

Lab session #10

Language Models

Dr Zied M'NASRI

Introduction

In this Lab session, we aim to implement some applications of language modelling, such as word sequence prediction using N-Gram model, document similarity measure using cosine distance, and context guessing using CBOW (Continuous Bag Of Words). To do the problems of this lab session, you need to use the following:

1. Python tutorial (see Lab session 1 material)
2. Colab tutorial (see Lab session 2 material)
3. Lecture 10 notes
4. Lab 10 example's notebook *Lab10_Example_N-gram_model.ipynb*
5. [Google's Colab](#) or Jupyter Notebook

Tutorial- How to build an N-Gram model

Upload the attached notebook *Lab10_Example_N-gram_model.ipynb* to Colab/Jupyter.

In this example, we show how to build a N-Gram model, particularly for $N=3$, i.e. a Tri-Gram model. We remind that an N-Gram model is useful to calculate, and predict, the probability of a any given sequence of N words in a given vocabulary. Therefore, this model will be based on a defined corpus, i.e. a large collection of texts, that provides the vocabulary.

Following the steps, a Tri-Gram is built using the “Reuters-News” corpus:

Step 1- Setup

Import the necessary libraries, including NLTK (Natural Language Tool Kit)

```
import numpy as np
import nltk
from nltk import bigrams, trigrams
from nltk.corpus import reuters
from collections import defaultdict
```

Step 2- Download text data

Download the NLTK resources, including the corpus “Reuters” and the tokenisation modules “punkt” and “nltk_punkt”

```
nltk.download('reuters')
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package reuters to /root/nltk_data...
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

Figure 1. Downloading the corpus and the tokenisation modules

Step 3- Tokenisation

Run the *tokenizer* on the corpus to convert the sentences into a sequence of tokens (separate units of text). We remind that a token can be a word, a sub-word or a punctuation mark. Tokenisation is necessary to convert the text into standard units.

```
words = nltk.word_tokenize(' '.join(reuters.words()))
words[:10]
```

```
['ASIAN', 'EXPORTERS', 'FEAR', 'DAMAGE', 'FROM', 'U', '.', 'S', '.-', 'JAPAN']
```

Figure 2. Visualisation of the first 10 tokens extracted from the “Reuters” corpus

Calculate and display the number of tokens (unique values)

```
print('number of words in the corpus =', len(words))
print('number of tokens (unique values) in the corpus =', len(set(words)))
```

```
number of words in the corpus = 1728932
number of tokens (unique values) in the corpus = 41516
```

Figure 3. Visualisation of the number of words and tokens in the “Reuters” corpus

Step 4- Creation of the tri-grams (in case N=3)

Create the list of all tri-grams contained in the corpus, and visualise the first 10 ones.

```
tri_grams = list(trigrams(words))
tri_grams[:10]
```

```
[('ASIAN', 'EXPORTERS', 'FEAR'),
 ('EXPORTERS', 'FEAR', 'DAMAGE'),
 ('FEAR', 'DAMAGE', 'FROM'),
 ('DAMAGE', 'FROM', 'U'),
 ('FROM', 'U', '.'),
 ('U', '.', 'S'),
 ('.', 'S', '-'),
 ('S', '-', 'JAPAN'),
 ('-', 'JAPAN', 'RIFT'),
 ('JAPAN', 'RIFT', 'Mounting')]
```

Figure 4. First 10 tri-grams extracted from the corpus

Step 5- Tri-gram initialisation

Initialise the tri-gram as a dictionary using the function `defaultdict()`

```
model = defaultdict(lambda: defaultdict(lambda: 0))
```

Step 6- Counting co-occurrences

Count the number of occurrences of each possible sequence of 3 tokens in the corpus

```
for w1, w2, w3 in tri_grams:
    model[(w1,w2)][w3] += 1
print(model)
# Display an example for a tri-gram starting by ('interest'+ 'and')
w0='interest'
w1='and'
model[w0,w1])
```

```
defaultdict(<function __main__.<lambda>.<locals>.<lambda>()>,
          {'concern': 1,
           'corporate': 2,
           'other': 1,
           'also': 1,
           'monetary': 1,
           'exchange': 2,
           'volumes': 1,
           'repayments': 1,
           'activity': 1,
           'principal': 2,
           'negotiations': 1,
           'liquidity': 2,
           'Pratt': 1,
           'control': 1,
           'holdings': 1})
```

Figure 5. Frequency of co-occurrence of tri-grams starting by the tokens (“*interest*”+ “*and*”)

Step 7- Calculating probabilities of co-occurrence

Transform the count of each tri-gram sequence into a probability. Remember that the N-gram model aims at calculating the probability of each sequence of N-words, and therefore will be used to predict the most likely word to complete the sequence.

```
for w1_w2 in model:
    total_count = float(sum(model[w1_w2].values()))
    for w3 in model[w1_w2]:
        model[w1_w2][w3] /= total_count
print(model)

# Display an example for a tri-gram starting by ('interest'+ 'and')
w0='interest'
w1='and'
model[w0,w1])
```

```
('interest',
 'and'): defaultdict(<function __main__.<lambda>.<locals>.<lambda>()>, {'concern': 0.05263157894736842,
 'corporate': 0.10526315789473684,
 'other': 0.05263157894736842,
 'also': 0.05263157894736842,
 'monetary': 0.05263157894736842,
 'exchange': 0.10526315789473684,
 'volumes': 0.05263157894736842,
 'repayments': 0.05263157894736842,
 'activity': 0.05263157894736842,
```

Figure 6. Tri-gram probabilities of the sequence of words starting by (“interest”+“and”)

Exercise 1- Word sequence prediction using a Tri-Gram model

In this example, we use the Tri-Gram model built in the tutorial to predict the most likely 3rd word given the first and the second words.

1. At the end of the code, enter 2 words and test the probability of the next word, as follows:

1.1. The words w_1 and w_2 are entered using the function `input()`

1.2. The next word, w_3 , is returned using the function `max()` applied to `model[(w1,w2)]` using as key the method `model[(w1,w2)].get`

1.3. Calculate the probability of the word w_3 using `model[(w1,w2)]`

2. Print the result, i.e. the word w_3 and its probability

3. Using the function `argpartition()` applied to the attribute values of `model[(w1,w2)]`, determine the indices of the 10 next words that have the highest probability to form the sequence (w_1, w_2, w_3) . Print the resulting words and their respective probabilities.

Hint! The function `argpartition()` is applied to lists, so you need to convert `model[(w1,w2)].values()` to a list

```
The 10 most likely word occurring after the sequence {'the', 'stock'} are :
was 0.025906735751295318
in 0.031088082901554383
. 0.051813471502590636
market 0.08290155440414503
of 0.10880829015544034
for 0.08808290155440408
split 0.051813471502590636
to 0.04145077720207251
, 0.04663212435233158
is 0.03626943005181345
```

**Figure 1.1. The 10 most probable words and their probability values
in the tri-gram (the, stock, word3)**

Exercise 2- Document similarity measure using cosine distance

In this exercise, we aim to measure the similarity between documents (or sentences) using the cosine distance between the word embedding vectors. The embedding vectors can be calculated using the TF-IDF (Term-Frequency Inverse-Document-Frequency) or the Sentence-Transformer model.

2.1. Setup

- Install `streamlit` using the command `!pip install`
- Import the following libraries:

```
import numpy as np
import streamlit as st
import gensim
from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.spatial.distance import cosine
from sentence_transformers import SentenceTransformer
from gensim.models import KeyedVectors
```

- Enter two sentences, `sentence1` and `sentence2`, e.g.

Enter the first sentence: The growth of AI is faster than ever
Enter the second sentence: AI is growing at a very rapid pace

Figure 2.1. Example of input sentences

- Calculate the TF-IDF based similarity measure between `sentence1` and `sentence2` as follows:

a. TF-IDF vectorisation: Declare a vectorizer variable as the output of the method `TfidfVectorizer()`

b. TF-IDF embedding: Calculate the TF-IDF matrix as the output of the method `vectorizer.fit_transform()` which input is the array `[sentence1, sentence2]`

c. Cosine distance: Calculate the cosine distance between the TF-IDF embeddings of `sentence1` and `sentence2` using the function `cosine()`

Hint! Each sentence embedding is a vector of the TF-IDF matrix, e.g. the embedding of sentence 1 is obtained as `tfidf_matrix[0].toarray()[0]`, whereas for sentence2, the embedding is `tfidf_matrix[1].toarray()[0]`

d. Similarity measure: Calculate and print the similarity measure as `1 - cosineDistance`

2.4. sentence Transformer-based similarity measure

- a. Sentence Transformer model declaration:** Declare a sentence Transformer model, `modelName`, as the output of the method `SentenceTransformer()` whose input is the BERT tokens base `"bert-base-nli-mean-tokens"`
- b. Embedding by encoding:** Calculate the embedding matrix, called `embeddingName`, as the output of the method `modelName.encoder()` which input is the array `[sentence1, sentence2]`
- c. Cosine distance:** Calculate the cosine distance using the function `cosine()` applied to the embedding vectors, i.e. `([embeddingName[0], embeddingName[1]])`
- d. Similarity measure:** Calculate and print the similarity measure as `1 - cosineDistance`

2.5. Print the results of both embedding model, i.e. TF-IDF and sentence-Transformer. Looking to the lecture 10 notes, try to understand why sentence-Transformer is more successful at capturing the similarity between both sentences.

```
TF-IDF Similarity: 0.155929
sentence-Transformer Similarity: 0.863134
```

Figure 2.2. Similarity measure results by TF-IDF and Sentence-Transformer word embedding models

Exercise 3- Prediction of the context with CBOW (Continuous Bag-of-Word)

In this exercise, we aim to detect the context using the CBOW model. Given a corpus of sentences, we train the CBOW model to cluster the words based on their contextual similarity. This visualisation of the clusters of words can be made using a projection of the model's results on a 2-D plane, using a technique called PCA (Principal Component Analysis).

1. Setup: Import the necessary libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, Lambda
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import to_categorical
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
```

2. Define a corpus: Enter an array of sentences, e.g.

```
corpus = [
    'The cat sat on the mat',
    'The dog ran in the park',
    'The bird sang in the tree'
]
```

3. Tokenisation: Convert each text into a sequence of tokens using:

3.1. Declare a tokenizerName using the method `tokenizer()`

3.2. Extract the tokens by applying the method `tokenizer.fit_on_texts()` on the declared corpus

3.3. Display the extracted tokens by printing the results contained in `tokenizer.index_word`

```
Extracted tokens: {1: 'the', 2: 'in', 3: 'cat', 4: 'sat', 5: 'on', 6: 'mat', 7: 'dog', 8: 'ran', 9: 'park', 10: 'bird', 11: 'sang', 12: 'tree'}
```

Figure 3.1. Extracted tokens from the input corpus

4. Embedding: Convert the corpus into a sequence of integers by applying the method `tokenizer.texts_to_sequences()` on the input corpus, and print the result:

```
Integer sequence: [[1, 3, 4, 5, 1, 6], [1, 7, 8, 2, 1, 9], [1, 10, 11, 2, 1, 12]]
```

Figure 3.2. Embedded sequences

5. To apply the CBOW model, we need to define the size of the vocabulary, the length of the embedding vector and the size of the window i.e. the number of surrounding words (remember that CBOW determines the word comprised between n previous and n following words). These parameters can be as:

- Size of vocabulary = `length(extracted tokens)+1` (Remember that extracted tokens are contained in `TokenizerName.word_index`)
- Embedding size = 10
- Window size = 2

6. Generate the context-target pairs: For each context, i.e. sequence of word, we calculate the target word that corresponds best:

6.1. Declare 2 empty arrays called `contexts` and `targets`

6.2. For each sequence in `sequences` (i.e. the extracted integer sequences) do :

For each word in the range(`window_size`, `len(sequence) - window_size`) do:

- The current context receives the sum of the sequence comprised between the position of the current word and position of (the current word – window size) and the sequence comprised between the position of next word and the position of (the next word + window_size)
- The current target receives the current sequence
- Append the contexts with the current context
- Append the targets with the current target

6.3. Print the results of the found contexts and targets

```
Contexts: [[1, 3, 5, 1], [3, 4, 1, 6], [1, 7, 2, 1], [7, 8, 1, 9], [1, 10, 2, 1], [10, 11, 1, 12]]
Targets: [4, 5, 8, 2, 11, 2]
```

Figure 3.3. Extracted contexts and targets

7. Prepare the extracted contexts and targets for training:

7.1. Convert the extracted contexts into a numpy array

```
[[ 1  3  5  1]
 [ 3  4  1  6]
 [ 1  7  2  1]
 [ 7  8  1  9]
 [ 1 10  2  1]
 [10 11  1 12]]
```

Figure 3.4. Array of extracted contexts

7.2. Convert the extracted targets into one-hot vectors where the width of the vector corresponds to the vocabulary size

Hint! Use the method `to_categorical()` using the vocabulary size as number of classes

```
[[0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```

Figure 3.5. Extracted targets converted to one-hot-vectors

8. Model building: Define the CBOW model as `sequential` and add the following layers:

- `embedding(input_dim=vocab_size, output_dim=embedding_size, input_length=2 * window_size)`
- `lambda(lambda x: tf.reduce_mean(x, axis=1))`
- `dense(units=vocab_size, activation='softmax')`

9. Modl visualisation: Display the model network using the method

```
modelName.summary()
```

Model: "sequential"		
Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lambda (Lambda)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)
Total params: 0 (0.00 B)		
Trainable params: 0 (0.00 B)		
Non-trainable params: 0 (0.00 B)		

Figure 3.6. CBOW model design**10. Model compilation:** Compile the model using the method `modelName.compile()` with the following parameters:

```
optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy']
```

11. Training: Train the model using the following parameters:

```
X, y, epochs=100, verbose=1
```

Where X and y are the array of extracted contexts and the one-hot target vectors, respectively

Hints!

- Use the methods `modelName.fit()` for training
- Let the variable `history` receive the output of the function `modelName.fit()`, so that it can be used to visualise the loss curves and other parameters of the model after training
- By setting `verbose = 1`, you can visualise the evolution of accuracy and loss on the training set over the epochs:

```

Epoch 1/100
1/1 _____ 0s 45ms/step - accuracy: 0.8333 - loss: 2.1936
Epoch 2/100
1/1 _____ 0s 53ms/step - accuracy: 0.8333 - loss: 2.1883
Epoch 3/100
1/1 _____ 0s 54ms/step - accuracy: 0.8333 - loss: 2.1830
Epoch 4/100
1/1 _____ 0s 56ms/step - accuracy: 0.8333 - loss: 2.1777
Epoch 5/100
1/1 _____ 0s 55ms/step - accuracy: 0.8333 - loss: 2.1723
Epoch 6/100
1/1 _____ 0s 55ms/step - accuracy: 0.8333 - loss: 2.1669
Epoch 7/100
1/1 _____ 0s 58ms/step - accuracy: 0.8333 - loss: 2.1615
Epoch 8/100
1/1 _____ 0s 58ms/step - accuracy: 0.8333 - loss: 2.1561
Epoch 9/100
1/1 _____ 0s 133ms/step - accuracy: 0.8333 - loss: 2.1506
Epoch 10/100
1/1 _____ 0s 64ms/step - accuracy: 0.8333 - loss: 2.1451

```

Figure 3.7. Training of the CBOW model

12. Loss Visualisation: Plot the loss curve during training as the result of `history.history(['loss'])`

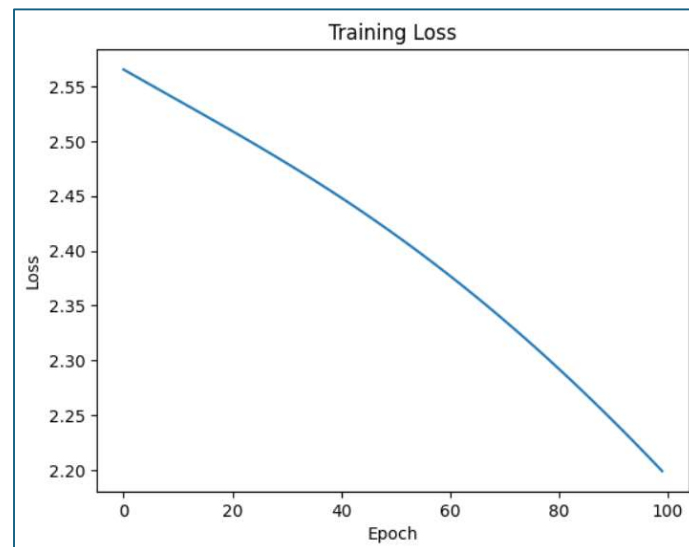


Figure 3.8. Training loss curve

13. Extract and display the predicted embeddings:

13.1. Declare an `embedding_layer` variable as the model's first layer (`modelName.layers[0]`)

13.2. Extract the trained embedding layer weights as the results of the method `embedding_layer.get_weights()[0]`

13.3. Print the extracted weights of the embedding layer

```
[[-0.03204536  0.01459307  0.01318209  0.02457781 -0.0404474  0.04790504
  0.0210781   0.03309401 -0.01180913 -0.03758812]
 [ 0.46586615 -0.40501308  0.24751128  0.01565773  0.05661799  0.13963637
  0.19302057  0.2901664  -0.299384   -0.48429346]
 [ 0.21390767 -0.32246163 -0.02117039 -0.32197288 -0.0357245  -0.1709015
  0.3041457  -0.00253851 -0.321525   -0.24512611]
 [ 0.10655886 -0.25913173  0.31337196  0.13558789  0.32628784  0.16722928
 -0.19110306  0.04516441  0.00198877 -0.3146525 ]
 [-0.23490992 -0.17059638  0.2574693   0.22890979  0.24128187 -0.21520822
  0.28025988 -0.2608183  -0.17778048 -0.18066035]
 [ 0.22115701 -0.28308308  0.24866505 -0.24763696  0.22126576  0.2401982
 -0.19389088  0.16721632  0.27371255 -0.18910111]
```

Figure 3.9. Weights of the embedding layer

14. Word embedding visualisation: To visualise the word embedding learnt by the CBOW model, we reduce the size of the extracted weights from 10 vectors to 2, using the PCA technique.

14.1. Declare a variable `pca` as the output of the method `PCA(n_components)` with `n_components = 2`

14.2. Calculate the reduced-size embedding as the output of the function `pca.fit_transform()` which input is the extracted embedding weight matrix (see 13.2)

14.3. Print the reduced-size embedding weight matrix:

```
[[-0.19721723 -0.02666041]
 [-0.03283731  0.38226646]
 [-0.43586192  0.0860678 ]
 [-0.1903379   0.25450116]
 [-0.43228605 -0.4665592 ]
 [-0.26908162  0.5985409 ]
 [-0.41850537 -0.42045256]
 [-0.02653426 -0.10452313]
 [ 0.40047765 -0.06684516]
 [ 0.42870635 -0.07001392]
 [ 0.35645825  0.02987326]
 [ 0.42006263 -0.09081438]
 [ 0.39695677 -0.1053807 ]]
```

Figure 3.10. Reduced embedding weight matrix by PCA

15. CBOW embedding visualisation

Visualise the list of token project on a 2-D plane using their reduced embedding weight matrix.

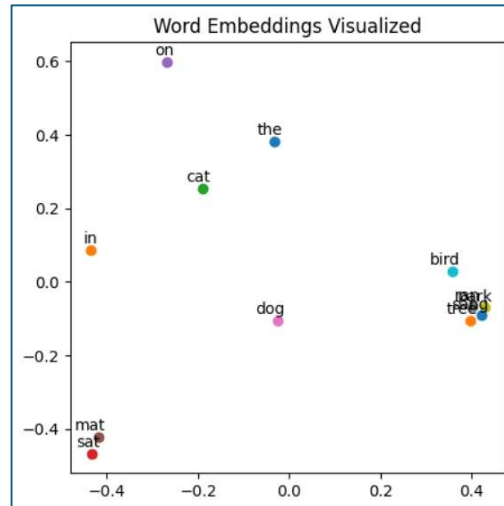


Figure 3.11. CBOW embedding: The words belonging to the same context (sequence) are closer to each other